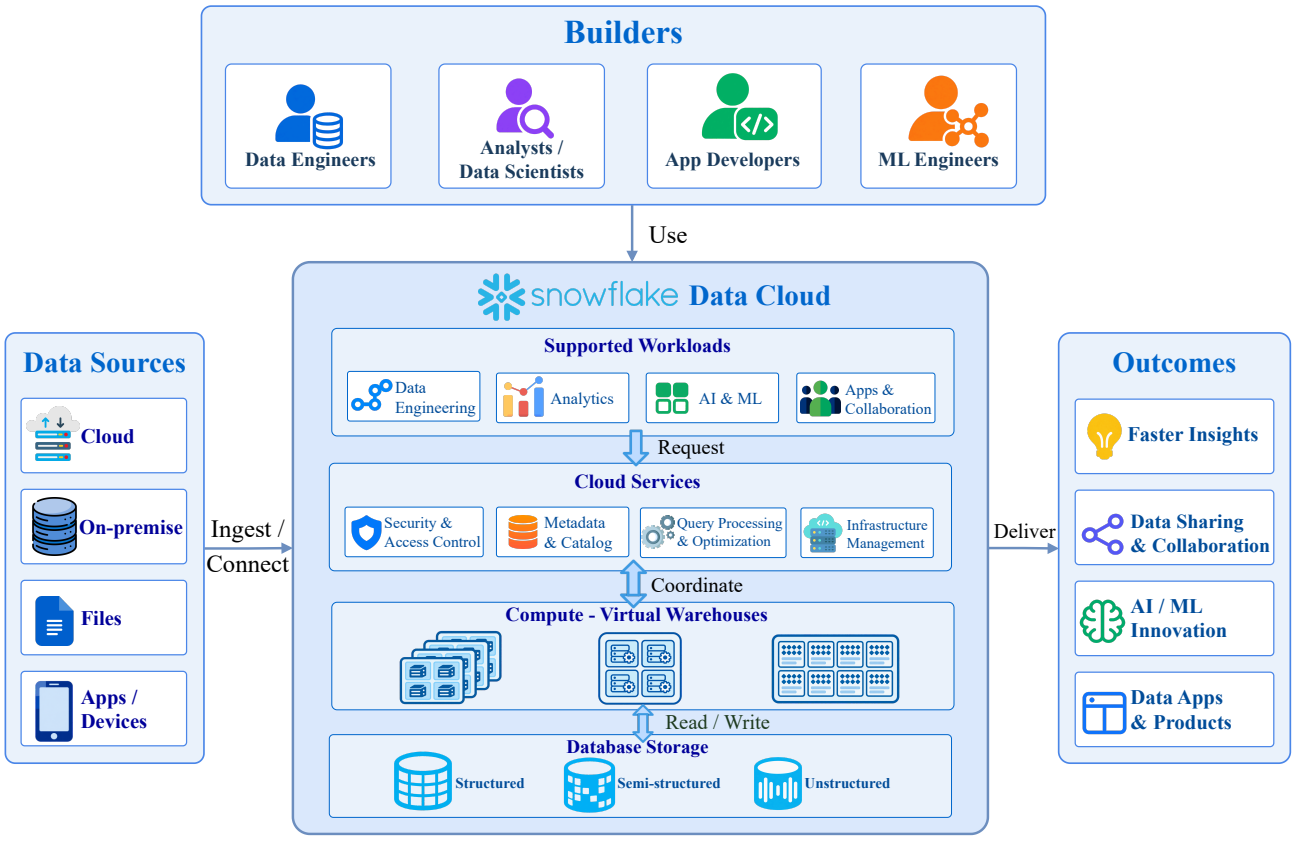
Modern Data Architecture with Snowflake and Apache Iceberg

(Data Platform)

Nguyễn Đăng Nhã **Trần Quang Minh** **Đinh Quang Vinh**

I. Giới thiệu

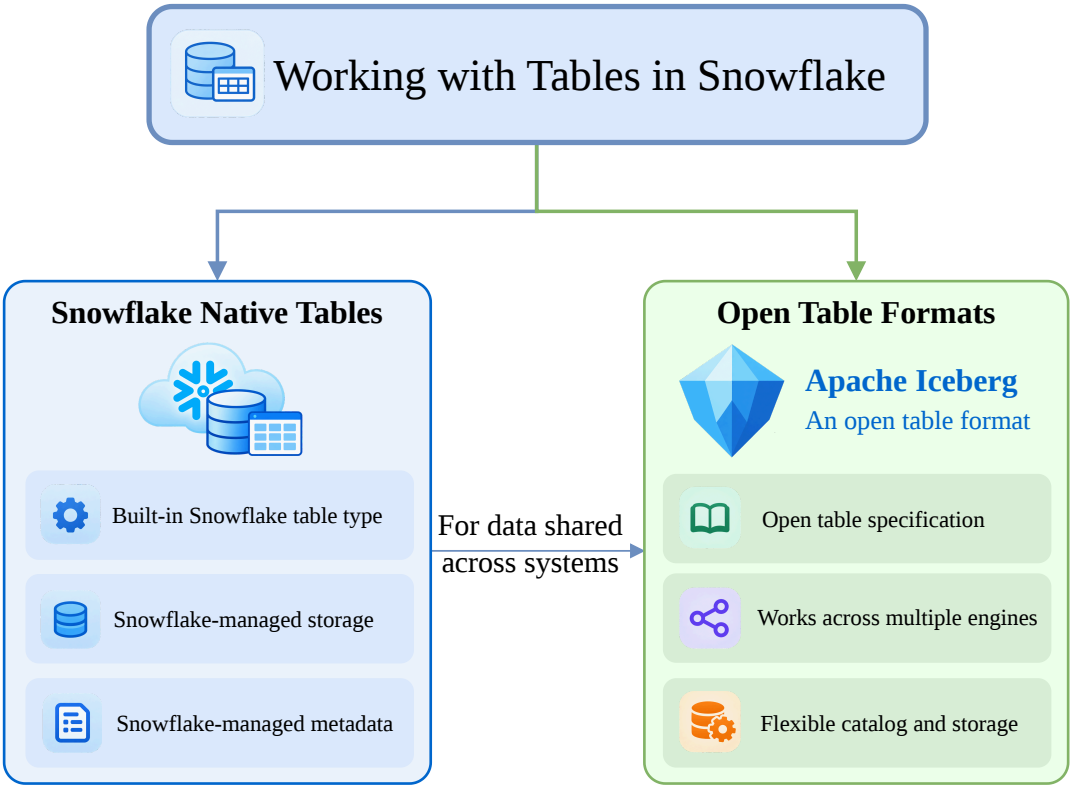
Các hệ thống dữ liệu hiện nay phải xử lý nhiều loại dữ liệu và workload khác nhau, từ dữ liệu có cấu trúc, bán cấu trúc đến phi cấu trúc, với quy mô có thể thay đổi đáng kể tùy bài toán. Snowflake được xây dựng để đáp ứng các nhu cầu này bằng cách tách **Storage**, **Compute** và **Cloud Services** thành những thành phần riêng, cho phép chúng mở rộng độc lập nhưng vẫn phối hợp trong cùng một nền tảng.



Hình 1: Tổng quan kiến trúc Snowflake Data Cloud.

Trong Snowflake, người dùng thường làm việc với dữ liệu thông qua tables, nơi dữ liệu được biểu diễn dưới dạng các hàng và cột quen thuộc. Khi mới làm quen với nền tảng, **Native Tables** thường là loại table được tiếp cận đầu tiên vì chúng đã được tích hợp sẵn trong Snowflake. Phía sau cách biểu diễn đơn giản này, Snowflake tự xử lý phần lớn việc chia nhỏ, lưu trữ và tổ chức dữ liệu để phục vụ truy vấn, còn người dùng chủ yếu tập trung vào việc tạo table, nạp dữ liệu và chạy query.

Khi cùng một dataset cần được nhiều hệ thống hoặc processing engine sử dụng, việc mỗi nền tảng yêu cầu một cách quản lý dữ liệu riêng có thể kéo theo các bước chuyển đổi, ingest lại hoặc duy trì thêm các bản dữ liệu phục vụ từng hệ thống. Trong bối cảnh đó, **Open Table Formats** cung cấp một cách chung để nhiều hệ thống có thể hiểu và làm việc với cùng một table.



Hình 2: Native Tables và Open Table Formats trong Snowflake, minh họa với Apache Iceberg.

Các Open Table Formats phổ biến hiện nay gồm Apache Iceberg, Delta Lake và Apache Hudi. Trong bài này, ta sẽ tập trung vào **Apache Iceberg** vì format này có thể làm việc với nhiều processing engine và hệ thống catalog khác nhau, đồng thời được Snowflake hỗ trợ trực tiếp trong nhiều khâu, từ quản lý table đến kết nối với các hệ thống bên ngoài.

Mục lục

I. Giới thiệu **1**

II. Native Tables và Open Table Formats trong Snowflake **4**

II.1. Cách Snowflake lưu trữ Native Tables 4

II.2. Storage, Table Format, Catalog và Query Engine 6

II.3. Iceberg: metadata, trạng thái và Catalog 8

III. Project: Data Warehouse với Snowflake Native Tables và Apache Iceberg 12

III.1. Mục tiêu và phạm vi bài toán 12

III.2. Thiết kế pipeline đối chiếu 13

III.3. Kế thừa project cũ và phần cần triển khai mới 14

III.4. Thiết lập AWS, S3 và kết nối Snowflake 14

III.5. Đối soát dữ liệu và kiểm thử pipeline 21

III.6. Đánh đổi về Storage, Compute và tính năng 21

III.7. Phương pháp đo hiệu năng và credits 22

III.8. Nhận xét lựa chọn kiến trúc 24

Phụ lục **25**

II. Native Tables và Open Table Formats trong Snowflake

II.1. Cách Snowflake lưu trữ Native Tables

Với Native Tables, người dùng làm việc với dữ liệu ở mức hàng và cột, còn Snowflake tự quản lý phần lớn cách dữ liệu được tổ chức ở lớp lưu trữ vật lý. Ở lớp này, dữ liệu không nhất thiết được lưu lần lượt theo từng hàng như cách chúng ta nhìn thấy trên table. Thay vào đó, các giá trị của từng cột có thể được lưu nối tiếp nhau. Cách tổ chức này được gọi là **Columnar**. Nhờ đó, những truy vấn chỉ cần một số cột có thể tập trung đọc đúng phần dữ liệu cần thiết.

Bên cạnh cách sắp xếp dữ liệu, một table lớn cũng thường được chia thành các phần nhỏ hơn để mỗi truy vấn không phải đọc toàn bộ table. Các phần này có thể hiểu đơn giản là **Partition**. Trong Snowflake, việc chia dữ liệu được thực hiện tự động bằng micro-partitions, nên người dùng không cần tự xác định ranh giới của từng partition.

Micro-partition

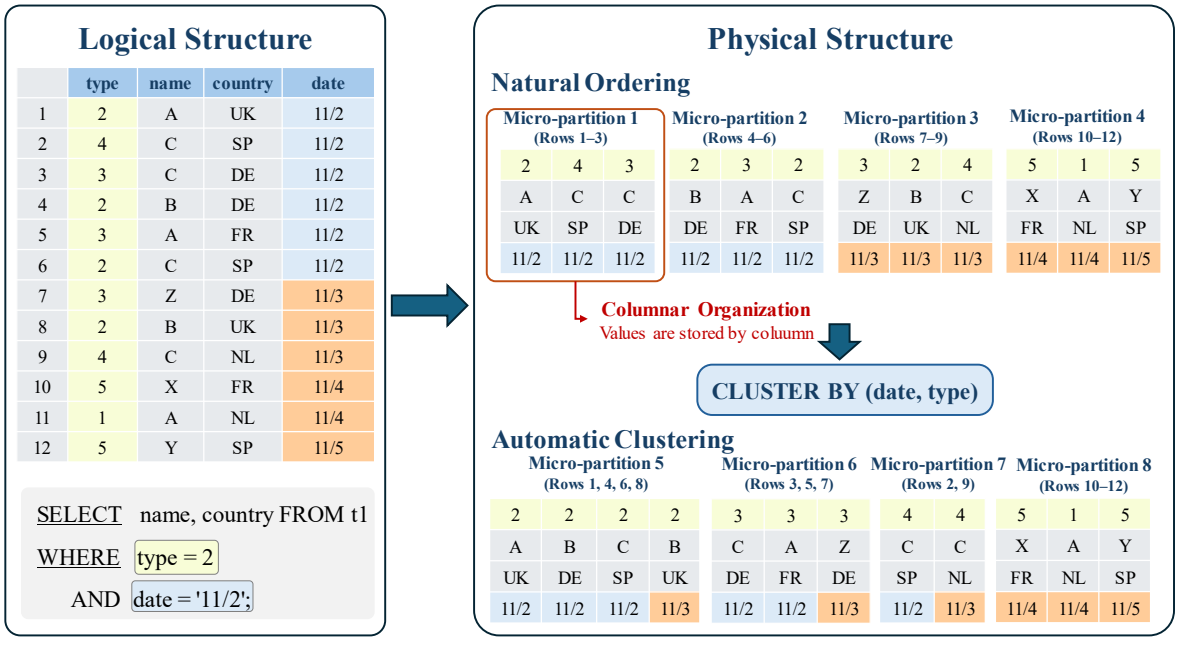
Là đơn vị lưu trữ vật lý được Snowflake tự động tạo và quản lý cho dữ liệu trong table.

Khi dữ liệu được nạp vào table, các hàng được phân bố vào các micro-partition và ban đầu thường phản ánh thứ tự dữ liệu đi vào hệ thống.

Natural Ordering

Là cách dữ liệu ban đầu được phân bố vào các micro-partition theo thứ tự nạp dữ liệu.

Trong Hình 3, các hàng của Table T1 được phân bố lần lượt từ Micro-partition 1 đến các micro-partition tiếp theo theo thứ tự nạp, thể hiện trạng thái phân bố ban đầu của dữ liệu.



Hình 3: Cách dữ liệu của Native Table được phân bố giữa các micro-partition.

Snowflake duy trì metadata cho từng micro-partition, chẳng hạn phạm vi giá trị nhỏ nhất và lớn nhất của các cột, số lượng giá trị khác nhau và một số thông tin thống kê liên quan. Metadata này giúp hệ thống biết trước micro-partition nào có khả năng chứa dữ liệu phù hợp với truy vấn.

Partition Pruning

Là quá trình Snowflake bỏ qua những micro-partition được xác định là không thể chứa dữ liệu phù hợp với điều kiện truy vấn.

Ví dụ, nếu một micro-partition chỉ chứa dữ liệu ngày từ "2026-01-01" đến "2026-01-10", truy vấn tìm dữ liệu của ngày "2026-03-15" có thể bỏ qua micro-partition đó mà không cần đọc dữ liệu bên trong. Tuy nhiên, hiệu quả pruning còn phụ thuộc vào cách dữ liệu được phân bố giữa các micro-partition. Trong Hình 3, với Natural Ordering, các hàng có type bằng 2 và date bằng "11/2" vẫn nằm trên cả Micro-partition 1 và Micro-partition 2, nên Snowflake phải đọc dữ liệu từ cả hai. Nếu table thường được truy vấn theo các cột này, người dùng có thể xác định Clustering Key để tổ chức dữ liệu phù hợp hơn. Việc lựa chọn key còn có thể xét đến cardinality, tức mức độ đa dạng giá trị của cột.

Clustering Key

Là một cột, nhóm cột hoặc biểu thức được chọn làm cơ sở để Snowflake tổ chức dữ liệu giữa các micro-partition theo cách table thường được truy vấn.

Trong Hình 3, hai cột date và type được chọn làm Clustering Key. Sau khi dữ liệu được tổ chức lại theo hai thuộc tính này, các hàng có type bằng 2 được tập trung vào Micro-partition 5. Với cùng truy vấn trước đó, Snowflake chỉ cần đọc ít micro-partition hơn thay vì phải đọc cả Micro-partition 1 và Micro-partition 2 như ở Natural Ordering.

Cách tổ chức dữ liệu theo Clustering Key giúp giảm mức độ chồng lấn về phạm vi giá trị giữa các micro-partition. Mức độ clustering có thể được đánh giá qua **Average Overlaps** và **Average Depth**; giá trị càng thấp thường cho thấy dữ liệu được cluster càng tốt. Khi query lọc theo Clustering Key, Snowflake có thể loại bỏ nhiều micro-partition không liên quan, qua đó giảm lượng dữ liệu cần đọc. Tuy nhiên, khi table tiếp tục nhận thêm dữ liệu hoặc được cập nhật, cách phân bố này có thể dần thay đổi, khiến một vùng giá trị liên quan đến nhiều micro-partition hơn và làm giảm hiệu quả pruning.

Automatic Clustering

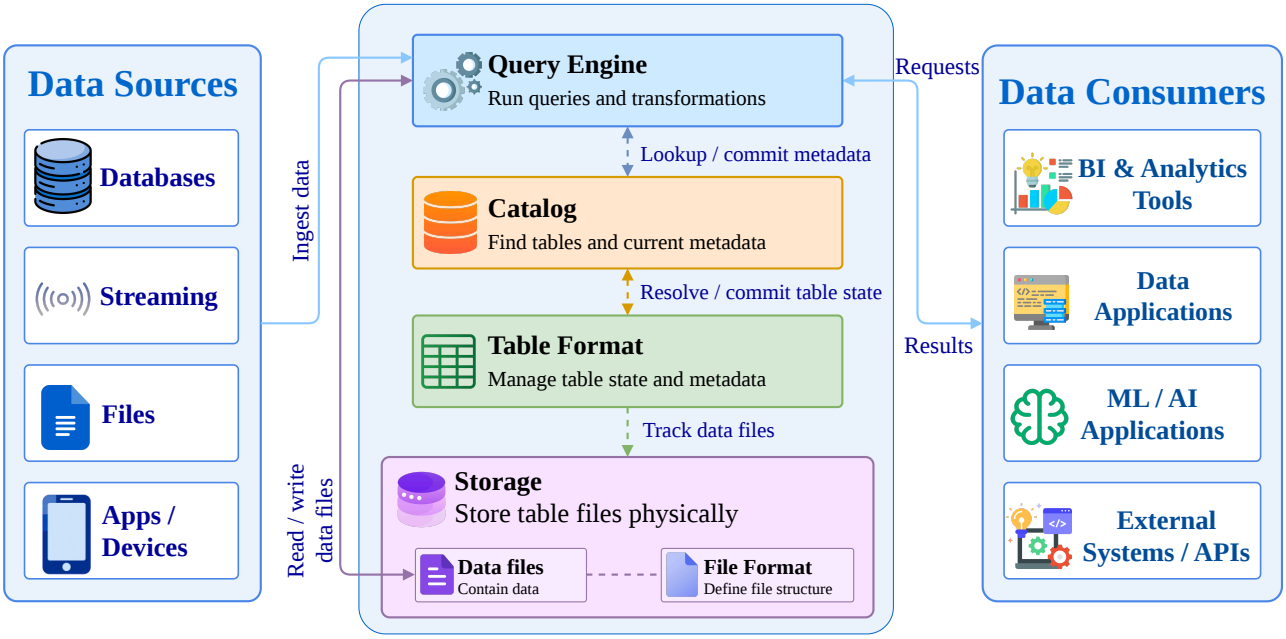
Là dịch vụ của Snowflake tự động theo dõi và duy trì cách tổ chức dữ liệu theo Clustering Key đã được xác định.

Khi cần, Snowflake tổ chức lại những micro-partition bị ảnh hưởng thay vì xử lý lại toàn bộ table. Nhờ đó, dữ liệu tiếp tục được phân bố phù hợp với Clustering Key, mức độ chồng lấn được kiểm soát và Partition Pruning tiếp tục hoạt động hiệu quả khi table thay đổi theo thời gian.

II.2. Storage, Table Format, Catalog và Query Engine

Khi dữ liệu và cách quản lý table gắn chặt với một nền tảng cụ thể, việc sử dụng cùng một dataset trên nhiều hệ thống có thể kéo theo các bước sao chép, chuyển đổi hoặc ingest lại dữ liệu. Duy trì riêng các bản dữ liệu trên nhiều hệ thống khiến việc chia sẻ và đồng bộ khó khăn hơn, từ đó tạo ra các vùng dữ liệu tách biệt, hay Data Silos. Việc phụ thuộc vào cách lưu trữ, quản lý và xử lý dữ liệu của một nhà cung cấp cũng khiến việc thay đổi nền tảng hoặc thay thế từng thành phần khó khăn hơn. Vấn đề này thường được gọi là Vendor Lock-in.

Trong phạm vi quản lý table, kiến trúc dữ liệu có thể được xem qua bốn thành phần chính gồm **Storage, Table Format, Catalog và Query Engine**. Mỗi thành phần đảm nhiệm một phần khác nhau trong việc lưu trữ, tổ chức, quản lý và xử lý dữ liệu. Cách nhìn này cũng giúp tiếp cận data lakehouse, nơi các thành phần có thể được triển khai độc lập hơn nhưng vẫn phối hợp với nhau thông qua các format và interface chung.



Hình 4: Mối liên hệ giữa Storage, Table Format, Catalog và Query Engine.

II.2.1. Storage, Data Files và File Format

Để hiểu cách một table được lưu trữ và quản lý, trước hết cần phân biệt nơi các file của table được lưu với cách dữ liệu bên trong từng file được tổ chức.

Storage

Là nơi các file của table được lưu vật lý.

Tùy hệ thống, Storage có thể là filesystem (hệ thống tệp), distributed storage (lưu trữ phân tán) hoặc object storage (lưu trữ đối tượng). Trong kiến trúc lakehouse, object storage thường được dùng để lưu các file tạo nên một table, bao gồm những file chứa dữ liệu và những file lưu thông tin dùng để mô tả hoặc quản lý table. Những file trực tiếp chứa dữ liệu của table được gọi là Data Files.

Data File

Là file vật lý chứa dữ liệu của table.

Việc hai Data Files cùng chứa dữ liệu dạng bảng không có nghĩa dữ liệu bên trong chúng phải được biểu diễn giống nhau. Cách biểu diễn này phụ thuộc vào format mà mỗi file sử dụng.

File Format

Quy định cách dữ liệu được tổ chức và biểu diễn bên trong một file.

Tùy File Format, dữ liệu có thể được bố trí theo hàng hoặc theo cột, chia thành các vùng nhỏ hơn và sử dụng những cơ chế như compression (nén dữ liệu) hoặc lưu statistics (thông tin thống kê về dữ liệu) để hỗ trợ quá trình đọc. Chẳng hạn, Parquet sắp xếp dữ liệu theo cột (columnar), nhờ đó một query chỉ sử dụng một số cột có thể hạn chế lượng dữ liệu phải đọc.

II.2.2. Table Format

Chỉ có Storage, Data Files và File Format vẫn chưa đủ để xác định một table. Khi dữ liệu nằm trên nhiều file, hệ thống còn phải biết file nào đang thuộc table, cấu trúc hiện tại ra sao và những thay đổi trước đó được ghi nhận như thế nào.

Table Format

Quy định cách nhiều Data Files được quản lý như một table thống nhất và cách trạng thái của table thay đổi theo thời gian.

Những thông tin dùng để mô tả trạng thái của table thường được gọi là **Metadata**, chẳng hạn schema hiện tại và tập Data Files đang thuộc table. Khi table thay đổi, metadata cũng được cập nhật để phản ánh trạng thái mới. Cách quản lý này giúp data lake hỗ trợ những workload cần các đặc tính thường thấy ở database hoặc data warehouse, chẳng hạn giao dịch tuân thủ ACID, snapshot (trạng thái của table tại một thời điểm), Schema Evolution (thay đổi schema theo thời gian) và Partition Evolution (thay đổi cách partition dữ liệu theo thời gian).

II.2.3. Catalog

Khi số lượng table tăng lên, hệ thống cần một cách để tìm table theo tên và xác định metadata hiện tại của table đó thay vì phải dò tìm trực tiếp các file trên Storage.

Catalog

Giúp hệ thống tổ chức và tìm các table, đồng thời xác định metadata hiện tại của từng table.

Ngoài vai trò tra cứu này, một số Catalog còn hỗ trợ **Namespace (không gian tên)** để tổ chức các table hoặc **Access Control (kiểm soát quyền truy cập)**. Nhờ đó, Catalog có thể trở thành điểm quản lý chung cho việc định danh, tổ chức và kiểm soát cách các table được truy cập trong hệ thống.

II.2.4. Query Engine

Storage, Table Format và Catalog giúp dữ liệu được lưu, mô tả và tìm thấy, nhưng bản thân chúng chưa thực hiện các query của người dùng. Công việc xử lý dữ liệu được đảm nhiệm bởi Query Engine.

Query Engine

Là thành phần tiếp nhận và thực thi các query hoặc phép biến đổi dữ liệu.

Khi làm việc với một table, Query Engine có thể tra cứu Catalog để tìm metadata hiện tại, từ đó xác định các Data Files cần đọc trên Storage. Với thao tác ghi, engine có thể tạo Data Files mới và cập nhật metadata theo quy tắc của Table Format. Query Engine cũng lựa chọn cách thực thi query, chẳng hạn phần dữ liệu cần đọc hoặc thứ tự xử lý các phép toán. Quá trình này thường được gọi là **Query Optimization**.

II.3. Iceberg: metadata, trạng thái và Catalog

Ở phần trước, Storage, Table Format và Catalog được tách thành những thành phần đảm nhiệm các vai trò khác nhau trong việc lưu trữ và quản lý table. Các Data Files trên Storage chưa đủ để xác định một table, bởi hệ thống còn cần biết đâu là trạng thái hiện tại và những file nào thuộc về trạng thái đó. Khi table được ghi, cập nhật hoặc thay đổi cấu trúc, tập Data Files này cũng có thể thay đổi theo thời gian.

Với Apache Iceberg, trạng thái của table được ghi nhận thông qua một cấu trúc gồm **Metadata File**, **Snapshot**, **Manifest List** và **Manifest File**. Catalog giúp xác định metadata hiện tại của table. Các phần tiếp theo giải thích vai trò và mối liên hệ giữa những thành phần này.

II.3.1. Từ Data Files đến Manifest Files

Trong data lake, dữ liệu có thể tồn tại ở dạng structured, semi-structured hoặc unstructured và được lưu trên các hệ thống như object storage. Khi một phần dữ liệu được tổ chức theo dạng bảng để phục vụ phân tích, dữ liệu của table có thể được phân bố trên nhiều Data Files. Khi số lượng file tăng lên, hệ thống cần xác định file nào thuộc table và file nào cần được đọc cho từng query. Apache Iceberg dùng metadata cho việc này. Manifest File trực tiếp theo dõi các Data Files và thông tin mô tả chúng.

Manifest File

Là metadata file theo dõi một nhóm Data Files và lưu các thông tin mô tả từng file.

Với mỗi Data File, Manifest File có thể lưu vị trí của file, số hàng và các statistics theo column như giá trị nhỏ nhất, lớn nhất hoặc số lượng giá trị null. Nhờ vậy, hệ thống có thể đánh giá một file có khả năng chứa dữ liệu phù hợp với điều kiện query hay không mà chưa cần đọc nội dung bên trong. Chẳng hạn, nếu metadata cho biết Data File A chỉ chứa dữ liệu của cột Order Date từ "2026-01-01" đến "2026-01-10", query tìm dữ liệu của ngày "2026-03-15" có thể bỏ qua file này ngay từ đầu. Việc sử dụng metadata để loại bỏ những Data Files chắc chắn không thể chứa dữ liệu phù hợp được gọi là **File Pruning**.

Sau nhiều lần ghi, table có thể tích lũy nhiều Data Files nhỏ, làm tăng số file mà hệ thống phải quản lý và đọc. Khi cần, Iceberg có thể kết hợp hoặc ghi lại chúng thành các file có kích thước phù hợp hơn mà không làm thay đổi dữ liệu logic của table. Thao tác này thường được gọi là **Compaction**.

II.3.2. Manifest List và pruning nhiều cấp

Khi số lượng Manifest Files tăng lên, việc mở từng manifest để kiểm tra sẽ tạo thêm chi phí. Manifest List là lớp metadata phía trên, giúp Iceberg thu hẹp phạm vi cần kiểm tra trước khi đi sâu xuống từng file.

Manifest List

Là metadata file tập hợp nhiều Manifest Files và lưu các thông tin tóm tắt giúp xác định những manifest có khả năng liên quan đến query.

Nhờ các summary ở cấp manifest, một số Manifest Files có thể được bỏ qua ngay từ đầu. Nếu Data File sử dụng Parquet, dữ liệu bên trong file còn được chia thành các nhóm hàng nhỏ hơn gọi là **Row Groups**. Mỗi nhóm có thể có statistics riêng để tiếp tục loại bỏ những phần dữ liệu không phù hợp.

Nhìn tổng thể, query có thể được thu hẹp lần lượt bằng cách bỏ qua Manifest Files, Data Files và Row Groups không liên quan. Có thể xem đây là ba cấp query pruning. Cơ chế này có cùng mục tiêu với Partition Pruning của Snowflake Native Tables: tránh đọc dữ liệu không cần thiết. Iceberg dựa trên metadata của Table Format kết hợp với statistics của Data Files, còn Snowflake Native Tables dựa trên metadata của micro-partitions do Snowflake quản lý.

II.3.3. Snapshot và trạng thái của table

Các thành phần metadata ở trên giúp Iceberg xác định những Data Files tạo nên table, nhưng tập file này có thể thay đổi sau mỗi lần ghi hoặc cập nhật dữ liệu. Vì vậy, Iceberg còn cần biểu diễn từng trạng thái của table theo thời gian để phân biệt dữ liệu trước và sau mỗi thay đổi.

Snapshot

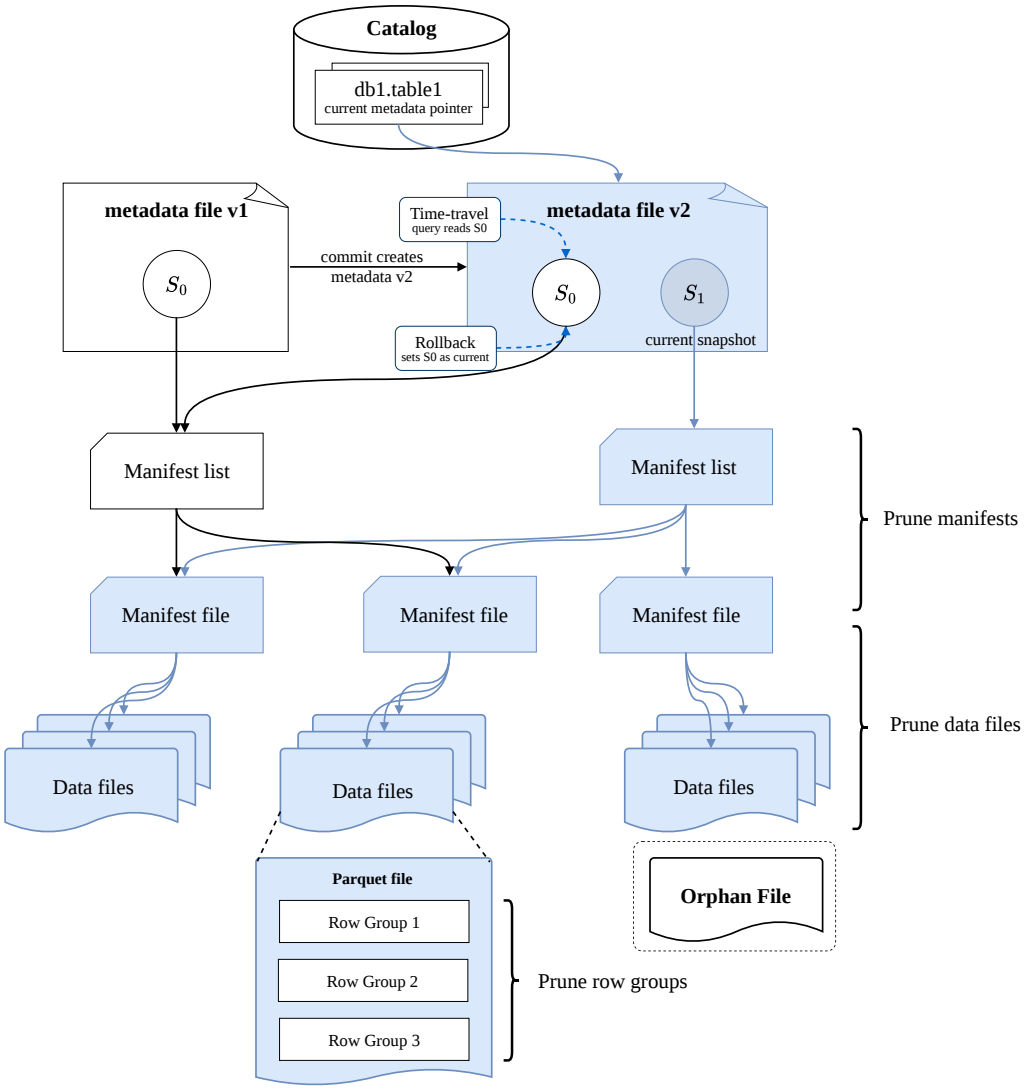
Biểu diễn trạng thái dữ liệu của một Iceberg Table tại một thời điểm.

Chẳng hạn, Snapshot S0 có thể đại diện cho trạng thái gồm Data File A và Data File B, trong khi Snapshot S1 được tạo sau đó có thể bổ sung thêm Data File C. Snapshot cũ không bị sửa trực tiếp để trở thành snapshot mới. Một snapshot giữ nguyên trạng thái sau khi được tạo. Đặc tính này thường được gọi là **Immutability**.

Với các thao tác như UPDATE, DELETE hoặc MERGE, Iceberg có thể xử lý phần dữ liệu thay đổi theo nhiều cách. **Copy-on-Write** ghi lại các Data Files bị ảnh hưởng khi thay đổi xảy ra, còn **Merge-on-Read** giữ lại các Data Files hiện có và lưu riêng phần thay đổi để kết hợp khi đọc. Trong cả hai trường hợp, kết quả sau cùng được ghi nhận bằng một snapshot mới.

Việc giữ lại các snapshot trước đó giúp Iceberg theo dõi table đã thay đổi như thế nào theo thời gian. Đây là nền tảng cho **Time Travel**, dùng để đọc lại một trạng thái trước đó, và **Rollback**,

dùng để đưa table về một snapshot cũ. Khi lịch sử tăng lên, những snapshot không còn cần thiết có thể được loại bỏ bằng thao tác Snapshot Expiration. Những file vẫn tồn tại trên Storage nhưng không còn được metadata của table tham chiếu được gọi là Orphan Files và có thể được dọn dẹp bằng thao tác Orphan File Removal.



Hình 5: Liên kết giữa Snapshot, dữ liệu table và các cấp query pruning trong Apache Iceberg.

II.3.4. Metadata File và sự thay đổi của table

Snapshot giúp Iceberg theo dõi trạng thái dữ liệu của table, nhưng hệ thống còn phải quản lý những thông tin ở cấp table như schema và cách dữ liệu được partition. Những thông tin này được lưu trong Metadata File, thường ở dạng JSON, để table có thể tiếp tục được mô tả nhất quán khi cấu trúc hoặc cách tổ chức dữ liệu thay đổi.

Metadata File

Lưu metadata ở cấp table và những thông tin cần thiết để mô tả table qua các phiên bản khác nhau.

Schema Evolution Iceberg gán cho mỗi field một ID riêng thay vì chỉ nhận diện column bằng tên hoặc vị trí. Chẳng hạn, field Name có Field ID là 2. Nếu field này được đổi tên thành Full Name, Field ID vẫn là 2, nên Iceberg nhận biết đây là cùng một field. Cách quản lý này hỗ trợ Schema Evolution, cho phép thêm, xóa, đổi tên hoặc sắp xếp lại column mà trong nhiều trường hợp không cần rewrite toàn bộ Data Files cũ.

Hidden Partitioning Bên cạnh schema, Metadata File còn phải theo dõi cách dữ liệu được chia thành các partition vì cách tổ chức này ảnh hưởng đến việc xác định phần dữ liệu cần đọc. Trong Iceberg, những quy tắc đó được biểu diễn bằng Partition Spec.

Partition Spec

Mô tả quy tắc dùng để tạo partition values từ các column của table.

Partition Spec có thể áp dụng các **Partition Transform**, chẳng hạn Month Transform hoặc Day Transform trên cột Order Date. Với cơ chế Hidden Partitioning, người dùng vẫn query trực tiếp trên cột gốc, ví dụ lọc Order Date theo giá trị "2026-03-15", còn Iceberg tự dựa vào Partition Spec để xác định dữ liệu cần đọc mà không yêu cầu người dùng biết cách partition vật lý.

Partition Evolution Cách partition cũng có thể thay đổi trong vòng đời của table. Dữ liệu cũ có thể sử dụng Month Transform với Partition Spec ID 0, trong khi dữ liệu mới sử dụng Day Transform với Partition Spec ID 1. Iceberg giữ lại các version của Partition Spec trong metadata và ghi nhận spec tương ứng với từng nhóm Data Files. Nhờ đó, dữ liệu cũ tiếp tục dùng cách partition ban đầu trong khi dữ liệu mới áp dụng cách mới mà không cần rewrite toàn bộ table. Khả năng thay đổi cách partition theo thời gian này được gọi là Partition Evolution.

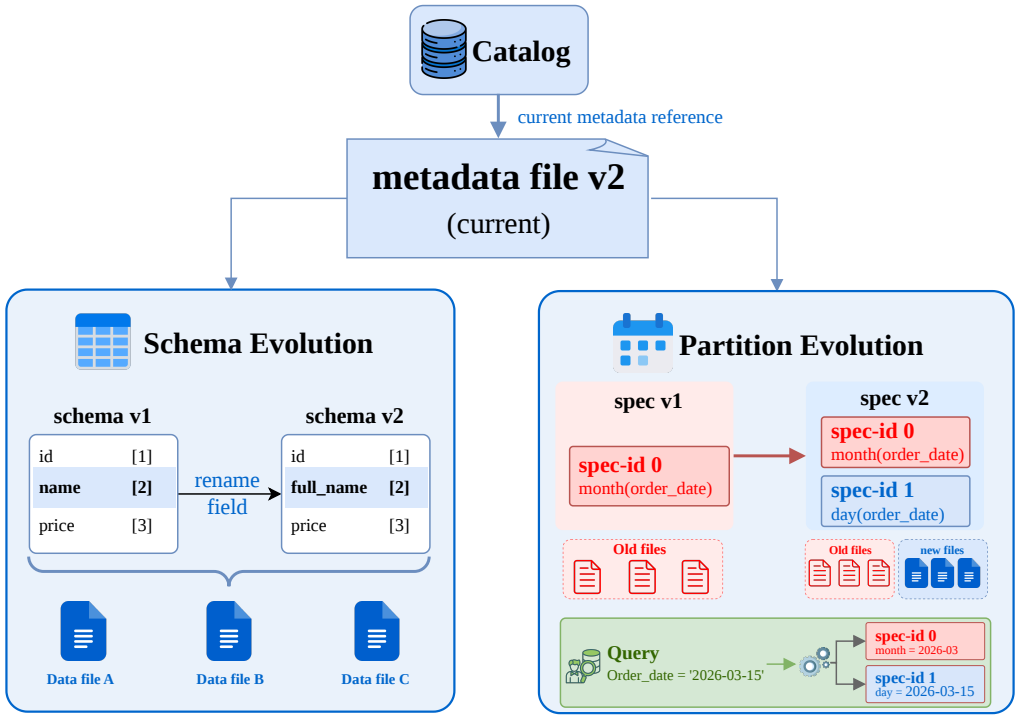
Ngoài schema và partitioning, Metadata File còn lưu những thông tin khác ở cấp table. Sort Orders mô tả cách dữ liệu có thể được sắp xếp khi ghi, Snapshots và Snapshot Log theo dõi các trạng thái của table theo thời gian, còn Table Properties lưu các cấu hình của table. Metadata File nhờ đó mô tả cấu trúc hiện tại và giữ những thông tin cần thiết để Iceberg quản lý cách table được tổ chức và thay đổi theo thời gian.

II.3.5. Catalog và metadata hiện tại của table

Khi table tiếp tục thay đổi, nhiều version của Metadata File có thể cùng tồn tại trên Storage. Catalog giúp hệ thống tìm table và xác định **Current Metadata Pointer**, tức con trỏ tới Metadata File đang đại diện cho trạng thái hiện tại. Trong ví dụ ở Hình 5, Catalog quản lý table db1.table1 và sử dụng con trỏ này để truy cập đúng trạng thái của table mà không phải tự dò tìm các file trên Storage. Khi table có trạng thái mới, Iceberg tạo Metadata File V2 từ Metadata File V1. Sau khi metadata mới đã sẵn sàng, Catalog thực hiện **Commit** bằng cách chuyển Current Metadata Pointer sang Metadata File V2.

Current Metadata Pointer được cập nhật trong một bước hoàn chỉnh để bảo đảm **Atomicity**, nên reader chỉ nhìn thấy trạng thái của Metadata File V1 hoặc Metadata File V2 mà không thấy table ở trạng thái chuyển tiếp. Nếu hai writer cùng bắt đầu từ Metadata File V1, writer đầu tiên có thể Commit Metadata File V2 trước, còn writer thứ hai phải kiểm tra lại Current Metadata Pointer để tránh ghi đè thay đổi vừa được ghi nhận. Cơ chế này được gọi là **Optimistic Concurrency Control**. Cùng với Snapshot và tính Atomicity của Commit, cơ chế này giúp Iceberg hỗ trợ **ACID Transactions**.

Iceberg hỗ trợ nhiều Catalog như Hive Metastore, REST Catalog và Apache Polaris. Dù khác nhau về cách triển khai và truy cập, các Catalog này vẫn sử dụng cùng một mô hình tổ chức metadata của Iceberg.



Hình 6: Metadata File lưu schema, Partition Specs và thông tin metadata ở cấp table, trong khi Catalog xác định metadata hiện tại.

III. Project: Data Warehouse với Snowflake Native Tables và Apache Iceberg

III.1. Mục tiêu và phạm vi bài toán

Phần project kế thừa bài toán **Vietnam Air Quality Data Pipeline** đã triển khai với Snowflake Native Tables. Pipeline tích hợp dữ liệu lịch sử chất lượng không khí từ Kaggle CSV với dữ liệu mới từ WAQI API, sau đó chuẩn hóa và cung cấp dữ liệu cho các câu hỏi phân tích: AQI thay đổi như thế nào theo thành phố, thời gian và mùa; thành phố hoặc trạm đo nào ghi nhận mức ô nhiễm cao; và xu hướng tổng hợp nào phù hợp để đưa lên dashboard.

Ở phiên bản trước, các lớp `Bronze`, `Silver` và `Gold` đều là Native Tables. Bản mở rộng không xây lại pipeline từ đầu: giữ nguyên nguồn dữ liệu, quy tắc làm sạch, mô hình Star Schema, Data

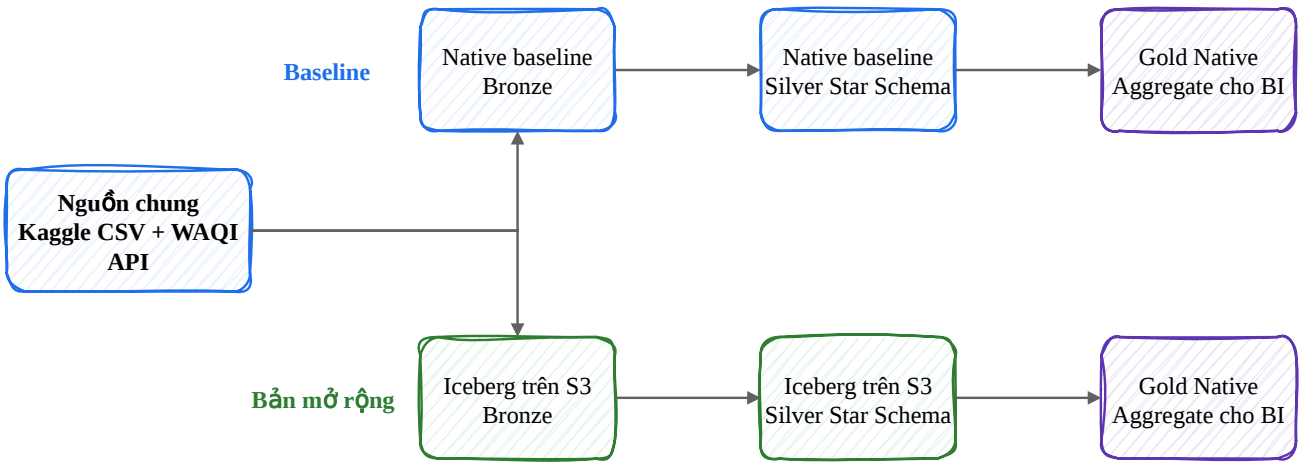
Quality Gate và các truy vấn aggregate; phần cần triển khai mới tập trung vào S3, IAM, external volume và cách tạo/nạp Iceberg Tables. Nhờ vậy, hai phương án được đối chiếu trên cùng một cơ sở nghiệp vụ.

III.2. Thiết kế pipeline đối chiếu

Phương án Native đóng vai trò baseline: cả ba lớp được lưu trong Snowflake và được Snowflake tự tổ chức thành micro-partitions. Phương án Iceberg sử dụng S3 làm nơi lưu Data Files dạng Parquet và metadata của Iceberg; trong thiết kế lai dùng cho project này, `Bronze` và `Silver` là Iceberg Tables, còn `Gold` tiếp tục là Native Tables để các aggregate nhỏ phục vụ BI trong Snowflake.

Lớp	Baseline Native	Phương án Iceberg lai
Nguồn	Kaggle CSV và WAQI API/seed data	Giữ nguyên nguồn và nội dung dữ liệu
`Bronze`	Native Tables giữ dữ liệu nguồn	Iceberg Tables trên S3, bảo toàn payload hoặc trường nguồn
`Silver`	Native Star Schema: `FACT_AQI` và bốn Dimension	Iceberg Star Schema với cùng grain, khóa và quy tắc chuẩn hóa
`Gold`	Native aggregate tables	Giữ Native để cung cấp các bảng tổng hợp cho dashboard
Query engine	Snowflake Virtual Warehouse	Cùng Snowflake Virtual Warehouse; engine khác có thể truy cập lớp Iceberg qua catalog được cấu hình

Luồng xử lý của phương án Iceberg gồm các bước: nạp dữ liệu nguồn vào `Bronze`; chuẩn hóa và tạo Fact/Dimension tại `Silver`; chạy Data Quality Gate; cuối cùng tổng hợp sang `Gold`. Hình minh họa đặt hai nhánh cạnh nhau: Native là baseline, còn Iceberg đổi nơi lưu `Bronze`/`Silver` sang S3 và vẫn giữ `Gold` trên Native để phục vụ dashboard.



Hình 7: Hai pipeline dùng chung dữ liệu và logic phân tích; bản mở rộng chuyển Bronze/Silver sang Iceberg trên S3, giữ Gold Native.

Thiết kế lai tập trung lợi ích của Iceberg vào các lớp có dung lượng và lịch sử tăng nhanh, đồng thời giữ lớp phục vụ dashboard trên Native Tables. Iceberg là định dạng mở nhưng mức độ độc lập với Snowflake còn phụ thuộc Catalog: Snowflake có thể quản lý Catalog trong khi Data Files nằm trên S3; hoặc có thể kết nối một Catalog bên ngoài thông qua catalog integration. Nếu dùng Snowflake làm Catalog, engine ngoài tương thích Iceberg REST có thể đọc và ghi qua Horizon Catalog với quyền truy cập do Snowflake kiểm soát, thay vì chỉ đọc trực tiếp các file trên bucket.[1–3]

III.3. Kế thừa project cũ và phần cần triển khai mới

Để người đọc biết rõ phạm vi cần làm, trước hết phân biệt phần giữ nguyên với phần phải viết hoặc cấu hình lại. Mục tiêu là thay lớp lưu trữ, không thay bài toán chất lượng không khí hay định nghĩa chỉ số phân tích.

Thành phần	Kế thừa từ Native	Thay đổi / làm mới cho Iceberg
Nguồn	Kaggle CSV, WAQI API, lịch nạp	Giữ nguyên; dùng cùng tập file để so sánh
Transform	Quy tắc parse, city mapping, grain và khóa	Giữ logic; đổi nguồn/đích trong các câu SQL và procedure
Silver	Fact, bốn Dimension và Star Schema	Tạo lại dưới dạng Iceberg Tables với cùng schema logic
DQ Gate	Kiểm tra null, miền AQI, freshness, khóa và coverage	Tái sử dụng; chạy trên bảng Iceberg trước khi tạo Gold
Gold / BI	Công thức aggregate và truy vấn dashboard	Giữ Native trong cấu hình lai; đối chiếu cùng query
AWS / Storage	Không có trong pipeline Native	Mới: CLI, S3, IAM role/policy, external volume
Benchmark	Bộ query nghiệp vụ	Bổ sung quy trình đo đồng nhất thời gian, bytes và credits

Vì vậy, không cần viết lại toàn bộ phép biến đổi hay tạo lại dữ liệu nguồn. Các bước cần làm mới là cấp quyền S3, khai báo Iceberg target, chuyển thao tác nạp và ghi vào Bronze/Silver, rồi xác minh rằng kết quả vẫn khớp baseline. Phần tiếp theo đi theo đúng thứ tự thực hiện; các đoạn code được đặt trong box riêng để có thể chạy và thay thế tham số theo môi trường.

III.4. Thiết lập AWS, S3 và kết nối Snowflake

Trong cấu hình này, Snowflake là nơi thực thi SQL và quản lý Catalog, còn S3 là nơi lưu trữ các file của Iceberg Table. Nếu project cần để AWS Glue hoặc một Iceberg REST Catalog khác làm Catalog chủ động, cần cấu hình thêm catalog integration và điều chỉnh cách tạo/refresh table. Hai mô hình này không nên bị gộp làm một: external volume kết nối đến Storage, còn Catalog quản lý tên table và metadata pointer.[1, 4]

III.4.1. Runbook triển khai theo thứ tự

Bước 1: Đăng nhập AWS CLI và tạo bucket Cài AWS CLI v2 theo hướng dẫn dành cho hệ điều hành đang dùng, sau đó xác nhận phiên bản bằng `aws -version`. Cần một AWS account và permission set được cấp quyền tạo bucket/IAM. Nên dùng IAM Identity Center để lấy temporary credentials thay vì ghi access key dài hạn vào project. Lệnh `aws configure sso` tạo named profile; `aws sts get-caller-identity` giúp kiểm tra account và role hiện tại trước khi tạo tài nguyên.[5, 6]

Bước 1: AWS CLI và S3 bucket

```
# Cau hinh profile mot lan; lam theo cac prompt SSO.
aws --version
aws configure sso --profile aq-iceberg
aws sso login --profile aq-iceberg
aws sts get-caller-identity --profile aq-iceberg

# Doi region va bucket name theo account/project cua ban.
export AWS_PROFILE=aq-iceberg
export AWS_REGION=ap-southeast-1
export BUCKET=aq-iceberg-yourname-unique-suffix

# us-east-1 khong truyen LocationConstraint; region khac thi phai truyen.
if [ "$AWS_REGION" = "us-east-1" ]; then
  aws s3api create-bucket --bucket "$BUCKET" --region "$AWS_REGION" \
    --object-ownership BucketOwnerEnforced
else
  aws s3api create-bucket --bucket "$BUCKET" --region "$AWS_REGION" \
    --create-bucket-configuration "LocationConstraint=$AWS_REGION" \
    --object-ownership BucketOwnerEnforced
fi

# Chan public access va bat versioning cho bucket.
aws s3api put-public-access-block --bucket "$BUCKET" \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,
    RestrictPublicBuckets=true
aws s3api put-bucket-versioning --bucket "$BUCKET" \
  --versioning-configuration Status=Enabled
aws s3api get-bucket-location --bucket "$BUCKET"
aws s3 ls "s3://$BUCKET/"
```

Thay `aq-iceberg-yourname-unique-suffix` bằng tên bucket còn trống. Chọn bucket cùng region với Snowflake account nếu khả thi. S3 bucket không cần tạo thư mục vật lý; các prefix như `iceberg/bronze/` sẽ được Snowflake tạo khi ghi table. Các lệnh trên tạo bucket tiêu chuẩn, không phải S3 directory bucket.[7]

Bước 2: Tạo IAM policy và role riêng cho Snowflake Profile AWS CLI ở bước trước là danh tính của người vận hành. Snowflake không dùng profile đó để truy cập S3: Snowflake assume một IAM role riêng. Policy dưới đây giới hạn quyền object vào prefix `iceberg/`; nếu bật SSE-KMS, cần bổ sung quyền KMS cho key tương ứng. Tên bucket không được chứa dấu chấm

vì Snowflake truy cập S3 bằng HTTPS virtual-hosted-style.

Tạo policy từ file JSON, sau đó gắn policy vào role. Ví dụ dùng CLI để tạo policy; role có thể được tạo trong IAM Console hoặc bằng lệnh tương ứng. Trust policy tạm thời chỉ dùng trong quy trình setup và cần được thay bằng Snowflake principal ngay sau khi tạo external volume. Nếu AWS STS chưa được kích hoạt ở region Snowflake sử dụng, cần bật STS cho region đó trong AWS Account settings trước khi xác minh volume.

Bước 2: IAM policy và role bootstrap

```
export ROLE_NAME=snowflake-aq-iceberg
export AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
# Tao mot lan, luu gia tri va dung lai trong CREATE EXTERNAL VOLUME.
export SNOWFLAKE_EXTERNAL_ID=$(openssl rand -hex 32)
printf 'External ID: %s\n' "$SNOWFLAKE_EXTERNAL_ID"

# Policy cho phep Snowflake quan ly object trong prefix iceberg/.
cat > s3-iceberg-policy.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:GetObjectVersion",
                 "s3:DeleteObject", "s3:DeleteObjectVersion"],
      "Resource": "arn:aws:s3:::$BUCKET/iceberg/*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::$BUCKET",
      "Condition": {"StringLike": {"s3:prefix": ["iceberg", "iceberg/*"]}}
    }
  ]
}
JSON
POLICY_ARN=$(aws iam create-policy --policy-name snowflake-aq-iceberg-s3 \
  --policy-document file://s3-iceberg-policy.json --query 'Policy.Arn' --output text)

# Trust tam thoi de Snowflake tao external volume va tra ve IAM principal.
cat > trust-bootstrap.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::$AWS_ACCOUNT_ID:root"},
    "Action": "sts:AssumeRole",
    "Condition": {"StringEquals": {"sts:ExternalId": "$SNOWFLAKE_EXTERNAL_ID"}}
  }]
}
JSON
aws iam create-role --role-name "$ROLE_NAME" \
  --assume-role-policy-document file://trust-bootstrap.json
```

```
aws iam attach-role-policy --role-name "$ROLE_NAME" --policy-arn "$POLICY_ARN"
aws iam get-role --role-name "$ROLE_NAME" --query 'Role.Arn' --output text
```

Lưu Role ARN vừa in ra để dùng ở Snowflake. Ở bước bootstrap, trust policy tạm thời cho phép Snowflake khởi tạo kết nối bằng External ID; ngay sau khi external volume trả về principal cụ thể, cần thay trust policy bằng ARN đó để thu hẹp quyền assume role.[8]

Bước 3: Tạo và xác minh external volume Chạy SQL bằng role Snowflake có quyền `CREATE EXTERNAL VOLUME`. Dùng Role ARN ở trên; hãy tạo một External ID ngẫu nhiên, lưu lại và dùng đúng cùng giá trị trong AWS trust policy. External volume xác định bucket/prefix và quyền ghi; Catalog của table sẽ được khai báo riêng. Tránh chạy lại lệnh tạo volume bằng `OR REPLACE` trong quy trình thông thường: nếu External ID thay đổi, trust policy trong AWS cũng phải được cập nhật.

Bước 3: Khai báo external volume

```
CREATE EXTERNAL VOLUME AQ_ICEBERG_VOL
  STORAGE_LOCATIONS = (
    (
      NAME = 'AQ_S3_LOCATION'
      STORAGE_PROVIDER = 'S3'
      STORAGE_BASE_URL = 's3://your-bucket-name/iceberg/'
      STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake-aq-iceberg'
      STORAGE_AWS_EXTERNAL_ID = 'aq-iceberg-unique-external-id'
    )
  )
  ALLOW_WRITES = TRUE;

DESC EXTERNAL VOLUME AQ_ICEBERG_VOL;
```

Thay bucket, Role ARN và External ID bằng giá trị thật. External ID trong SQL phải giống chính xác giá trị được sinh ở Bước 2. Từ kết quả `DESC EXTERNAL VOLUME`, lấy `STORAGE_AWS_IAM_USER_ARN`; thay principal bootstrap bằng ARN này trong AWS rồi mới chạy lệnh xác minh bên dưới.[8, 9]

Bước 3: Thu hẹp trust policy về principal Snowflake

```
# Gan ARN nay bang STORAGE_AWS_IAM_USER_ARN trong ket qua DESC EXTERNAL VOLUME.
export SNOWFLAKE_IAM_USER_ARN='arn:aws:iam::123456789001:user/snowflake-generated-user'
cat > trust-snowflake.json <<JSON
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "$SNOWFLAKE_IAM_USER_ARN"},
    "Action": "sts:AssumeRole",
    "Condition": {"StringEquals": {"sts:ExternalId": "$SNOWFLAKE_EXTERNAL_ID"}}
  }]
```

```
}
JSON
aws iam update-assume-role-policy --role-name "$ROLE_NAME" \
  --policy-document file://trust-snowflake.json
```

Bước 3: Kiểm tra quyền truy cập S3

```
SELECT SYSTEM$VERIFY_EXTERNAL_VOLUME('AQ_ICEBERG_VOL');
```

Bước 4: Tạo Iceberg Bronze và nạp dữ liệu Tạo database/schema riêng để giữ nguyên baseline. Ví dụ dưới đây giả định database project Native tên `AQ_NATIVE`; đổi prefix stage/file format nếu tên trong repo khác. Snowflake hỗ trợ Iceberg v3 ở trạng thái Generally Available; v3 hỗ trợ kiểu `VARIANT` để giữ JSON payload tương tự Bronze cũ. Chỉ chọn v3 nếu các engine bên ngoài cũng hỗ trợ v3, vì mọi reader và writer của cùng table cần tương thích với format version đó. Nếu ưu tiên tương thích với engine cũ, lưu JSON thành chuỗi hoặc cột typed và dùng version phù hợp.[10, 11]

Bước 4: Tạo database, schema và Iceberg Bronze

```
USE WAREHOUSE AQ_WH;

CREATE DATABASE IF NOT EXISTS AQ_ICEBERG;
CREATE SCHEMA IF NOT EXISTS AQ_ICEBERG.BRONZE;
CREATE SCHEMA IF NOT EXISTS AQ_ICEBERG.SILVER;
CREATE SCHEMA IF NOT EXISTS AQ_ICEBERG.GOLD;

CREATE ICEBERG TABLE AQ_ICEBERG.BRONZE.API_RAW (
  queried_city STRING,
  raw_payload VARIANT,
  ingested_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  source_file STRING
)
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'AQ_ICEBERG_VOL'
  BASE_LOCATION = 'bronze/api_raw'
  ICEBERG_VERSION = 3;
```

Ở project cũ, file JSON đã được đưa lên internal stage `AQ_NATIVE.BRONZE.AQ_INTERNAL_STAGE` và dùng file format `AQ_NATIVE.BRONZE.AQ_JSON_FORMAT`. Có thể tái sử dụng cùng source files để nạp vào Iceberg target; với JSON và CSV, Snowflake chuyển dữ liệu sang Parquet trong base location của table. Mỗi table cần `BASE_LOCATION` riêng, là đường dẫn tương đối với external volume.[12, 13]

Bước 4: Nạp JSON vào Iceberg Bronze

```
COPY INTO AQ_ICEBERG.BRONZE.API_RAW
  (queried_city, raw_payload, source_file)
FROM (
  SELECT
    $1:data:_queried_city::STRING,
    $1,
    METADATA$FILENAME
  FROM @AQ_NATIVE.BRONZE.AQ_INTERNAL_STAGE
)
PATTERN = '.*[.]json'
FILE_FORMAT = AQ_NATIVE.BRONZE.AQ_JSON_FORMAT
ON_ERROR = 'CONTINUE';
```

Tạo thêm `HISTORICAL_CSV` với các cột raw kiểu `STRING` như project trước rồi nạp CSV bằng file format tương ứng. Với pipeline chạy lại, giữ cơ chế idempotency qua `CONTROL.LOADED_FILES`; không coi việc chạy lại `COPY INTO` là thay thế cho kiểm soát file đã nạp. Không tự ghi Parquet vào prefix `iceberg/`: Snowflake phải quản lý Data Files và metadata qua Iceberg table để Catalog luôn nhất quán.

Bước 5: Tạo Silver Iceberg và chuyển logic biến đổi Tạo năm Iceberg tables ở Silver: `FACT_AQI`, `DIM_DATE`, `DIM_TIME`, `DIM_CITY` và `DIM_STATION`. Giữ nguyên schema logic, grain, khóa, quy tắc city mapping và các phép kiểm tra của project cũ; mỗi table cần một `BASE_LOCATION` riêng. Ví dụ dưới đây minh họa khai báo Fact với các khóa mà Gold sử dụng; hãy bổ sung những cột còn lại theo schema Native của project và tạo bốn Dimension theo cùng mẫu.

Bước 5: Tạo Iceberg Fact Table

```
CREATE ICEBERG TABLE AQ_ICEBERG.SILVER.FACT_AQI (
  date_key NUMBER(8, 0),
  time_key NUMBER(4, 0),
  city_key NUMBER(38, 0),
  station_key NUMBER(38, 0),
  measured_at TIMESTAMP_NTZ,
  aqi DOUBLE,
  source STRING
)
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'AQ_ICEBERG_VOL'
  BASE_LOCATION = 'silver/fact_aqi'
  ICEBERG_VERSION = 3;
```

Trong procedure nạp Silver đã có, giữ phần union nguồn CSV/API, đồng bộ Dimension bằng `MERGE` và nối surrogate key như cũ; thay các stream/table nguồn bằng bảng Iceberg Bronze và thay đích thành các Iceberg tables ở Silver. Đoạn SQL sau chỉ minh họa phần trích xuất API payload, không phải toàn bộ procedure; phần union CSV, chuẩn hóa và join bốn Dimension được kế thừa từ project Native.

Bước 5: Parse API payload sang Silver

```
SELECT
  r.raw_payload:data:idx::INT          AS waqi_idx,
  r.raw_payload:data:aqi::DOUBLE       AS aqi,
  r.raw_payload:data:city:name::STRING AS station,
  r.queried_city                       AS city_slug,
  r.ingested_at,
  'api'                                AS source
FROM AQ_ICEBERG.BRONZE.API_RAW AS r;
```

Bước 6: Chạy lại Data Quality Gate Tái sử dụng Data Quality Gate cũ và chỉ đổi bảng đầu vào. Ví dụ kiểm tra nhanh dưới đây cần cho kết quả phù hợp với quy tắc chất lượng đã đặt ra; nếu không đạt, dừng trước khi tạo Gold. Các điều kiện city coverage, freshness và foreign-key integrity vẫn cần chạy trong procedure đầy đủ.

Bước 6: Kiểm tra chất lượng Fact AQI

```
SELECT
  COUNT(*) AS total_rows,
  COUNT_IF(aqi IS NULL) AS null_aqi,
  COUNT_IF(aqi < 0 OR aqi > 500) AS out_of_range_aqi
FROM AQ_ICEBERG.SILVER.FACT_AQI;
```

Bước 7: Tạo lại Gold Native Sau khi DQ đạt, Gold tiếp tục là Native Table để dashboard truy vấn thuận tiện. Truy vấn sau minh họa aggregate theo tháng/thành phố; thay tên bảng và công thức bằng các aggregate thật của project cũ. Các bảng Gold còn lại được tạo lại bằng truy vấn aggregate cũ, không cần chuyển thành Iceberg trong benchmark chính.

Bước 7: Tổng hợp Gold Native

```
CREATE OR REPLACE TABLE AQ_ICEBERG.GOLD.AQI_MONTHLY AS
SELECT
  c.city_slug,
  YEAR(d.full_date) AS year,
  MONTH(d.full_date) AS month,
  AVG(f.aqi) AS avg_aqi,
  MAX(f.aqi) AS max_aqi
FROM AQ_ICEBERG.SILVER.FACT_AQI AS f
JOIN AQ_ICEBERG.SILVER.DIM_CITY AS c
  ON c.city_key = f.city_key
JOIN AQ_ICEBERG.SILVER.DIM_DATE AS d
  ON d.date_key = f.date_key
GROUP BY c.city_slug, YEAR(d.full_date), MONTH(d.full_date);
```

Trước tiên chạy lần lượt Load, Silver, DQ và Gold bằng tay; chỉ bật Tasks sau khi kiểm tra thành công. Chỉ cấu hình Stream-triggered Task nếu loại table, Iceberg version, Catalog và nguồn phát

sinh thay đổi được hỗ trợ. Đặc biệt, khả năng theo dõi thay đổi do Snowflake ghi và thay đổi do engine ngoài ghi không mặc nhiên giống nhau; không copy nguyên Task DAG của Native mà chưa kiểm thử insert, delete, retry và schema change.[10, 14, 15]

III.5. Đối soát dữ liệu và kiểm thử pipeline

Trước khi đo hiệu năng, cần xác nhận rằng hai nhánh cho ra cùng kết quả nghiệp vụ. Các đối soát tối thiểu gồm số dòng của từng lớp, số thành phố và trạm đo, khoảng thời gian dữ liệu, số giá trị null, miền giá trị AQI, tính toàn vẹn khóa ngoại và kết quả của năm bảng aggregate. Các tiêu chí này tương ứng với Data Quality Gate trong project cũ; có thể tiếp tục ghi kết quả vào `CONTROL.DQ_RESULTS`.

Đối với dữ liệu thô JSON, cần thống nhất cách biểu diễn khi ghi vào Iceberg. Nếu giữ payload dưới dạng chuỗi JSON, phải kiểm tra rằng dữ liệu gốc vẫn truy vết được và Silver trích xuất đúng các trường đang dùng. Đối với CSV, cần giữ nguyên quy tắc ép kiểu và chuẩn hóa giữa hai nhánh. So sánh chỉ có ý nghĩa khi cả hai table chứa cùng một tập bản ghi, cùng schema logic và cùng thời điểm dữ liệu.

Phần orchestration cũng cần được kiểm tra theo đúng loại Iceberg Table và Catalog đang dùng. Khả năng dùng Streams khác nhau theo version và Catalog: với Iceberg v3 do Snowflake quản lý, có thể dùng standard và append-only Streams; với Iceberg v3 do Catalog bên ngoài quản lý, hỗ trợ standard và insert-only Streams nhưng không hỗ trợ append-only Streams. Standard Stream trên table external v3 còn phụ thuộc cách engine ghi row lineage. Trên table do Snowflake quản lý, append-only Stream không thể tiêu thụ thay đổi do engine ngoài ghi; một số dạng Stream trên table external v2 cũng không được hỗ trợ. Triggered Tasks có thể phối hợp với Streams trên Iceberg, nhưng pipeline nên chạy thử các trường hợp insert, update, delete, schema change và retry trước khi xem đây là thay thế trực tiếp cho cấu hình Native cũ.[10, 14, 15]

III.6. Đánh đổi về Storage, Compute và tính năng

Iceberg không đồng nghĩa với việc mọi thành phần đều rẻ hơn. Với external volume do người dùng quản lý, chi phí lưu data và metadata được tính ở cloud provider; ngoài dung lượng còn có thể có request, data transfer và egress. Snowflake vẫn tính compute của Virtual Warehouse khi chạy query hoặc biến đổi dữ liệu. Vì vậy, tổng chi phí cần được xem như tổng của Snowflake credits, S3 storage/request/transfer và chi phí vận hành Catalog, thay vì chỉ so sánh giá mỗi GB. Snowflake không cung cấp Fail-safe cho Iceberg Tables nằm trên storage do khách hàng quản lý; chính sách bảo vệ và phục hồi dữ liệu cần được thiết kế ở AWS.[1, 16]

Tiêu chí	Native Tables	Iceberg trên S3
Storage	Snowflake quản lý; phí lưu trữ theo cơ chế của Snowflake	AWS tính phí cho S3; chủ project quản lý lifecycle và recovery
Compute	Virtual Warehouse của Snowflake	Vẫn dùng Virtual Warehouse khi query trong Snowflake; engine khác có thể dùng compute riêng
Query	Tối ưu sâu với micro-partitions và metadata do Snowflake quản lý	Phụ thuộc file sizing, partitioning, metadata, vị trí Storage và query profile
Tính mở	Phụ thuộc nhiều hơn vào hệ sinh thái Snowflake	Parquet và Iceberg metadata có thể được dùng bởi nhiều engine tương thích, khi quyền và Catalog được cấu hình
Tính năng	Tích hợp trực tiếp với các tính năng của Snowflake	Hỗ trợ phụ thuộc Catalog, Iceberg version và nơi phát sinh thay đổi
Vận hành	Ít thành phần Storage/Catalog phải tự quản lý	Cần quản lý quyền S3, chi phí truyền dữ liệu, retention và dọn metadata/file

Về query, không nên khẳng định Iceberg luôn chậm hơn hoặc luôn tiêu tốn nhiều credits hơn Native Tables. Một query có thể khác nhau về thời gian chạy, bytes scanned, cache và lượng metadata cần xử lý; credits còn phụ thuộc kích thước warehouse và thời gian warehouse hoạt động. Do đó, kết luận về compute phải lấy từ workload và cấu hình thực tế của project, không suy ra chỉ từ việc table là Iceberg.

Time Travel cũng cần được diễn giải theo Catalog. Với table dùng Snowflake làm Catalog, Snowflake quản lý vòng đời metadata và snapshot theo retention đã cấu hình. Với table dùng external Catalog, thời gian truy vấn lịch sử của Snowflake bị giới hạn theo metadata retention và Snowflake không xóa snapshot/file trong S3 thay cho Catalog bên ngoài. Với cả hai mô hình, Iceberg snapshot không nên được xem là mặc nhiên tương đương hoàn toàn với Time Travel trên Native Table.[16]

III.7. Phương pháp đo hiệu năng và credits

Để phép so sánh có thể lặp lại, chạy các truy vấn nghiệp vụ tương đương trên hai nhánh: tổng hợp AQI theo ngày/tháng và thành phố; xếp hạng thành phố; phân tích theo giờ hoặc mùa; truy vấn station có AQI cao nhất. Dùng cùng kích thước warehouse, cùng khoảng dữ liệu, cùng số lần chạy và cùng concurrency. Ghi riêng lần chạy đầu và các lần chạy sau để phân biệt ảnh hưởng của cache; tắt result cache nếu cần đo lại thực thi query, và không gộp thời gian nạp/compaction vào thời gian truy vấn đọc.

Với mỗi query, ghi execution time, bytes scanned, partitions scanned/pruned, số dòng trả về và trạng thái cache từ Query Profile/Query History. Gắn hai nhóm workload bằng query tag hoặc dùng warehouse thử nghiệm riêng cho từng nhánh, không chạy workload khác trong lúc đo. Trong đoạn SQL dưới đây, chạy riêng từng lệnh đổi tag; sau mỗi lần đổi, chạy toàn bộ bộ

query benchmark trên cùng session. Đặt tag thành `AQ_REPORTING` trước khi chạy query đọc lịch sử để không tính chính query báo cáo vào benchmark. Credit có thể được đối soát từ lịch sử metering của warehouse; khi cần phân bổ đến từng query, dùng lịch sử attribution nếu khả dụng cho account. Không cộng idle time của warehouse vào chi phí query, nhưng phải báo riêng nếu muốn so sánh chi phí vận hành cả pipeline.[17, 18]

Kết quả lưu trữ nên ghi thêm tổng dung lượng Parquet và metadata tại S3, dung lượng tương ứng ở Native Tables, retention/snapshot đang giữ và số liệu chi phí S3 trong cùng khoảng thời gian. Vì chi phí S3 và Snowflake phụ thuộc region, retention, lượng dữ liệu, warehouse và bảng giá của account, project chỉ nên đưa ra số đo sau khi pipeline đã chạy trên cùng một dataset và cấu hình được ghi rõ.

Đo benchmark: gắn nhãn workload và đọc lịch sử

```
-- Gan tag de tach rieng workload trong Query History.
ALTER SESSION SET QUERY_TAG = 'AQ_NATIVE';
-- Chay rieng lenh nay, sau do chay bo query benchmark tren Native.
ALTER SESSION SET QUERY_TAG = 'AQ_ICEBERG';
-- Khi Native da xong, chay lenh nay rieng roi chay cung bo query tren Iceberg.
ALTER SESSION SET QUERY_TAG = 'AQ_REPORTING';
-- Chuyen tag truoc khi chay cac query doc lich su ben duoi.

SELECT
  query_tag,
  query_id,
  execution_time,
  bytes_scanned,
  partitions_scanned,
  partitions_total
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag IN ('AQ_NATIVE', 'AQ_ICEBERG')
  AND execution_status = 'SUCCESS'
ORDER BY start_time;

SELECT
  query_tag,
  SUM(credits_attributed_compute) AS compute_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY
WHERE query_tag IN ('AQ_NATIVE', 'AQ_ICEBERG')
GROUP BY query_tag;
```

Chạy trên warehouse thử nghiệm không có workload khác; nếu bảng Account Usage chưa cập nhật, đợi ingestion delay rồi chạy lại. Query attribution phân bổ compute credit cho query, còn phần idle/auto-suspend và chi phí toàn pipeline cần báo riêng.[18, 19]

Chỉ số cần báo cáo	Native baseline	Iceberg trên S3
Số dòng / kết quả DQ	Cùng tập dữ liệu	Đối soát với Native
Thời gian truy vấn	Median của các lượt đo	Median của các lượt đo
Bytes scanned / partitions scanned	Lấy từ Query Profile	Lấy từ Query Profile
Credits compute	Tách query khỏi idle time	Tách query khỏi idle time
Dung lượng lưu trữ	Snowflake storage usage	Dung lượng S3 và metadata
Chi phí bổ sung	Tính năng Snowflake liên quan	S3 request/transfer và vận hành Catalog

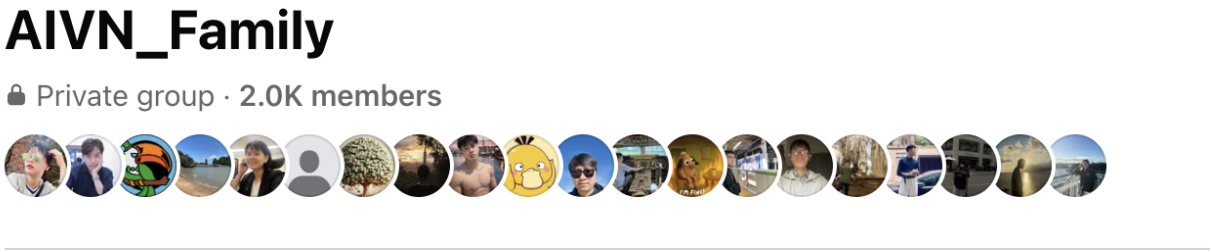
III.8. Nhận xét lựa chọn kiến trúc

Kết quả project nên được đọc theo workload. Native Tables phù hợp khi ưu tiên tích hợp sâu, vận hành đơn giản và truy vấn tối ưu trong Snowflake. Iceberg trên S3 phù hợp hơn khi cần chia sẻ các lớp dữ liệu lớn với hệ sinh thái engine khác, kiểm soát trực tiếp storage hoặc giảm phụ thuộc vào định dạng lưu trữ riêng của một nền tảng. Mô hình lai `Bronze/Silver Iceberg + Gold Native` là một điểm cân bằng: giữ các dữ liệu lớn ở định dạng mở, đồng thời duy trì lớp tổng hợp tiện dụng cho dashboard Snowflake.

Việc lựa chọn cuối cùng không dựa riêng vào giá S3 hay một lần chạy query. Cần cân nhắc đồng thời dung lượng và tốc độ tăng dữ liệu, tỷ lệ đọc/ghi, độ trễ chấp nhận được, engine cần chia sẻ, yêu cầu phục hồi, tính năng CDC/orchestration và kết quả benchmark. Nếu workload nhỏ, khoản tiết kiệm storage có thể không bù phần thiết lập và vận hành bổ sung; nếu dữ liệu lớn và được nhiều engine dùng chung, lợi ích về quyền kiểm soát và tính di động có thể quan trọng hơn chênh lệch hiệu năng.

Phụ lục

1. **Solution:** Các file code cài đặt hoàn chỉnh và phần trả lời nội dung trắc nghiệm có thể được tải tại đây.
2. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại đây. Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.



References

[1] Snowflake Documentation, *Apache iceberg tables*, https://docs.snowflake.com/en/user-guide/tables-iceberg, Accessed 15 October 2026.

[2] Snowflake Documentation, *Configuring a catalog integration for iceberg tables*, https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-catalog-integration, Accessed 15 October 2026.

[3] Snowflake Documentation, *Access apache iceberg tables with an external engine through snowflake horizon catalog*, https://docs.snowflake.com/en/user-guide/tables-iceberg-access-using-external-query-engine-snowflake-horizon, Accessed 16 October 2026.

[4] Snowflake Documentation, *Configuring an external volume for iceberg tables*, https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-external-volume, Accessed 15 October 2026.

[5] Amazon Web Services Documentation, *Installing or updating the latest version of the aws cli*, https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html, Accessed 16 October 2026.

[6] Amazon Web Services Documentation, *Configuring iam identity center authentication for the aws cli*, https://docs.aws.amazon.com/cli/latest/userguide/cli-configure-sso.html, Accessed 16 October 2026.

[7] Amazon Web Services Documentation, *Create-bucket: Aws cli command reference*, https://docs.aws.amazon.com/cli/latest/reference/s3api/create-bucket.html, Accessed 16 October 2026.

[8] Snowflake Documentation, *Configure an external volume for amazon s3*, https://docs.snowflake.com/en/user-guide/tables-iceberg-configure-external-volume-s3, Accessed 16 October 2026.

[9] Snowflake Documentation, *Create external volume*, https://docs.snowflake.com/en/sql-reference/sql/create-external-volume, Accessed 16 October 2026.

[10] Snowflake Documentation, *Iceberg table support and limitations*, https://docs.snowflake.com/en/user-guide/tables-iceberg-v3-specification-support, Accessed 16 October 2026.

[11] Snowflake Documentation, *Support for apache iceberg version 3 (general availability)*, https://docs.snowflake.com/en/release-notes/2026/other/2026-05-07-iceberg-v3-ga, Accessed 16 October 2026.

[12] Snowflake Documentation, *Create iceberg table*, https://docs.snowflake.com/en/sql-reference/sql/create-iceberg-table, Accessed 16 October 2026.

[13] Snowflake Documentation, *Load data into apache iceberg tables*, https://docs.snowflake.com/en/user-guide/tables-iceberg-load, Accessed 16 October 2026.

[14] Snowflake Documentation, *Streams on iceberg tables*, https://docs.snowflake.com/en/user-guide/streams-intro, Accessed 15 October 2026.

[15] Snowflake Documentation, *Triggered tasks*, https://docs.snowflake.com/en/user-guide/tasks-triggered, Accessed 15 October 2026.

[16] Snowflake Documentation, *Managing iceberg tables*, https://docs.snowflake.com/en/user-guide/tables-iceberg-manage, Accessed 15 October 2026.

[17] Snowflake Documentation, *Understanding compute cost*, https://docs.snowflake.com/en/user-guide/cost-understanding-compute, Accessed 15 October 2026.

[18] Snowflake Documentation, *Query_attribution_history view*, https://docs.snowflake.com/en/sql-reference/account-usage/query_attribution_history, Accessed 15 October 2026.

[19] Snowflake Documentation, *Query_history view*, https://docs.snowflake.com/en/sql-reference/account-usage/query_history, Accessed 16 October 2026.